

T. VIKAS

2303A52281

```
n = int(input())
arr = list(map(int, input().split()))
tree = [0] * (4 * n)

def gcd(a, b):
    while b != 0:
        a, b = a % b, b
    return a

def build(node, start, end):
    if start == end:
        tree[node] = arr[start]
        return
    mid = (start + end) // 2
    build(node * 2, start, mid)
    build(node * 2 + 1, mid + 1, end)
    tree[node] = gcd(tree[node * 2], tree[node * 2 + 1])

def update(node, start, end, index, value):
    if start == end:
        tree[node] = value
        return
    mid = (start + end) // 2
    if index <= mid:
        update(node * 2, start, mid, index, value)
    else:
        update(node * 2 + 1, mid + 1, end, index, value)
    tree[node] = gcd(tree[node * 2], tree[node * 2 + 1])

def query(node, start, end, left, right):
    if right < start or end < left:
        return 0
    if left <= start and end <= right:
```

```

        return tree[node]
    mid = (start + end) // 2
    a = query(node * 2, start, mid, left, right)
    b = query(node * 2 + 1, mid + 1, end, left, right)
    return gcd(a, b)
q = int(input())
for _ in range(q):
    operation = list(map(int, input().split()))
    if operation == 0:
        index = operation[1]
        value = operation[2]
        arr[index] = value
        update(1, 0, n - 1, value, index)
    else:
        left = operation[1]
        right = operation[2]
        print(1, 0, n - 1, left, right)

```

2nd question:

```

n = int(input())
arr = list(map(int, input().split()))
tree = [1] * (4 * n)
def build(node, start, end):
    if start == end:
        tree[node] = arr[start]
        return
    mid = (start + end) // 2
    build(node * 2, start, mid)
    build(node * 2 + 1, mid + 1, end)
    tree[node] = tree[node * 2] * tree[node * 2 + 1]

```

```

def update(node, start, end, index, value):
    if start == end:
        tree[node] = value
        return
    mid = (start + end) // 2
    if index <= mid:
        update(node * 2, start, mid, index, value)
    else:
        update(node * 2 + 1, mid + 1, end, index, value)
    tree[node] = tree[node * 2] * tree[node * 2 + 1]

def query(node, start, end, left, right):
    if right < start or end < left:
        return 1
    if left <= start and end <= right:
        return tree[node]
    mid = (start + end) // 2
    a = query(node * 2, start, mid, left, right)
    b = query(node * 2 + 1, mid + 1, end, left, right)
    return a * b

build(1, 0, n - 1)
q = int(input())
for _ in range(q):
    operation = list(map(int, input().split()))
    if operation[0] == 1:
        index = operation[1]
        value = operation[2]
        arr[index] = value
        update(1, 0, n - 1, index, value)

```

else:

left = operation[1]

right = operation[2]

print(query(1, 0, n - 1, left, right))

3rd Question:

n = int(input())

arr = list(map(int, input().split()))

tree = [0] * (4 * n)

def build(node, start, end):

if start == end:

tree[node] = arr[start]

return

mid = (start + end) // 2

build(node * 2, start, mid)

build(node * 2 + 1, mid + 1, end)

tree[node] = tree[node * 2] | tree[node * 2 + 1]

def update(node, start, end, index, value):

if start == end:

tree[node] = value

return

mid = (start + end) // 2

if index <= mid:

update(node * 2, start, mid, index, value)

else:

update(node * 2 + 1, mid + 1, end, index, value)

tree[node] = tree[node * 2] | tree[node * 2 + 1]

def query(node, start, end, left, right):

if right < start or end < left:

```

    return 0
if left <= start and end <= right:
    return tree[node]
mid = (start + end) // 2
a = query(node * 2, start, mid, left, right)
b = query(node * 2 + 1, mid + 1, end, left, right)
return a | b
build(1, 0, n - 1)
q = int(input())
for _ in range(q):
    operation = list(map(int, input().split()))
    if operation[0] == 1:
        index = operation[1]
        value = operation[2]
        arr[index] = value
        update(1, 0, n - 1, index, value)
    else:
        left = operation[1]
        right = operation[2]
        print(query(1, 0, n - 1, left, right))

```

4th question:

```

n = int(input())
arr = list(map(int, input().split()))
tree = [0] * (4 * n)
def build(node, start, end):
    if start == end:
        tree[node] = arr[start]
    return

```

```

mid = (start + end) // 2
build(node * 2, start, mid)
build(node * 2 + 1, mid + 1, end)
tree[node] = tree[node * 2] + tree[node * 2 + 1]
def update(node, start, end, index, value):
    if start == end:
        tree[node] = value
        return
    mid = (start + end) // 2
    if index <= mid:
        update(node * 2, start, mid, index, value)
    else:
        update(node * 2 + 1, mid + 1, end, index, value)
    tree[node] = tree[node * 2] + tree[node * 2 + 1]
def kth_one(node, start, end, k):
    if tree[node] < k:
        return -1
    if start == end:
        return start
    mid = (start + end) // 2
    if tree[node * 2] >= k:
        return kth_one(node * 2, start, mid, k)
    return kth_one(node * 2 + 1, mid + 1, end, k - tree[node * 2])
build(1, 0, n - 1)
q = int(input())
for _ in range(q):
    operation = list(map(int, input().split()))
    if operation[0] == 1:

```

```
index = operation[1]
```

```
value = operation[2]
```

```
arr[index] = value
```

```
update(1, 0, n - 1, index, value)
```

```
else:
```

```
k = operation[1]
```

```
print(kth_one(1, 0, n - 1, k))
```

5th code:

```
n = int(input())
```

```
arr = list(map(int, input().split()))
```

```
tree = [0] * (4 * n)
```

```
def build(node, start, end):
```

```
    if start == end:
```

```
        tree[node] = arr[start]
```

```
    return
```

```
    mid = (start + end) // 2
```

```
    build(node * 2, start, mid)
```

```
    build(node * 2 + 1, mid + 1, end)
```

```
    tree[node] = max(tree[node * 2], tree[node * 2 + 1])
```

```
def update(node, start, end, index, value):
```

```
    if start == end:
```

```
        tree[node] = value
```

```
    return
```

```
    mid = (start + end) // 2
```

```
    if index <= mid:
```

```
        update(node * 2, start, mid, index, value)
```

```
    else:
```

```

        update(node * 2 + 1, mid + 1, end, index, value)
    tree[node] = max(tree[node * 2], tree[node * 2 + 1])
def find_first(node, start, end, left, right, x):
    if end < left or start > right:
        return -1
    if tree[node] < x:
        return -1
    if start == end:
        return start
    mid = (start + end) // 2
    result = find_first(node * 2, start, mid, left, right, x)
    if result != -1:
        return result
    return find_first(node * 2 + 1, mid + 1, end, left, right, x)
build(1, 0, n - 1)
q = int(input())
for _ in range(q):
    operation = list(map(int, input().split()))
    if operation[0] == 1:
        index = operation[1]
        value = operation[2]
        arr[index] = value
        update(1, 0, n - 1, index, value)
    else:
        left = operation[1]
        right = operation[2]
        x = operation[3]
        print(find_first(1, 0, n - 1, left, right, x))

```

6th code:

```
n = int(input())
arr = list(map(int, input().split()))
tree = [0] * (4 * n)

def build(node, start, end):
    if start == end:
        if arr[start] == 0:
            tree[node] = 1
        else:
            tree[node] = 0
        return
    mid = (start + end) // 2
    build(node * 2, start, mid)
    build(node * 2 + 1, mid + 1, end)
    tree[node] = tree[node * 2] + tree[node * 2 + 1]

def update(node, start, end, index, value):
    if start == end:
        if value == 0:
            tree[node] = 1
        else:
            tree[node] = 0
        return
    mid = (start + end) // 2
    if index <= mid:
        update(node * 2, start, mid, index, value)
    else:
        update(node * 2 + 1, mid + 1, end, index, value)
    tree[node] = tree[node * 2] + tree[node * 2 + 1]
```

```

def query(node, start, end, left, right):
    if right < start or end < left:
        return 0
    if left <= start and end <= right:
        return tree[node]
    mid = (start + end) // 2
    a = query(node * 2, start, mid, left, right)
    b = query(node * 2 + 1, mid + 1, end, left, right)
    return a + b

build(1, 0, n - 1)

q = int(input())

for _ in range(q):
    operation = list(map(int, input().split()))
    if operation[0] == 1:
        index = operation[1]
        value = operation[2]
        arr[index] = value
        update(1, 0, n - 1, index, value)
    else:
        left = operation[1]
        right = operation[2]
        print(query(1, 0, n - 1, left, right))

```

7th code:

```

n = int(input())
arr = list(map(int, input().split()))
tree = [[0, 0, 0, 0] for _ in range(4 * n)]
# [left_ones, right_ones, best, length]

```

```

def build(node, start, end):
    if start == end:
        if arr[start] == 1:
            tree[node] = [1, 1, 1, 1]
        else:
            tree[node] = [0, 0, 0, 1]
        return
    mid = (start + end) // 2
    build(node * 2, start, mid)
    build(node * 2 + 1, mid + 1, end)
    merge(node)

def merge(node):
    left = tree[node * 2]
    right = tree[node * 2 + 1]
    left_ones = left[0]
    right_ones = right[1]
    best = max(left[2], right[2], left[1] + right[0])
    if left[0] == left[3]:
        left_ones = left[3] + right[0]
    if right[1] == right[3]:
        right_ones = right[3] + left[1]
    length = left[3] + right[3]
    tree[node] = [left_ones, right_ones, best, length]

def update(node, start, end, index, value):
    if start == end:
        if value == 1:
            tree[node] = [1, 1, 1, 1]

```

```

    else:
        tree[node] = [0, 0, 0, 1]
    return
mid = (start + end) // 2
if index <= mid:
    update(node * 2, start, mid, index, value)
else:
    update(node * 2 + 1, mid + 1, end, index, value)
merge(node)
def query(node, start, end, left, right):
    if right < start or end < left:
        return [0, 0, 0, 0]
    if left <= start and end <= right:
        return tree[node]
    mid = (start + end) // 2
    a = query(node * 2, start, mid, left, right)
    b = query(node * 2 + 1, mid + 1, end, left, right)
    if a[3] == 0:
        return b
    if b[3] == 0:
        return a
    left_ones = a[0]
    right_ones = b[1]
    if a[0] == a[3]:
        left_ones = a[3] + b[0]
    if b[1] == b[3]:
        right_ones = b[3] + a[1]
    best = max(a[2], b[2], a[1] + b[0])

```

```

length = a[3] + b[3]
return [left_ones, right_ones, best, length]
build(1, 0, n - 1)
q = int(input())
for _ in range(q):
    operation = list(map(int, input().split()))
    if operation[0] == 1:
        index = operation[1]
        value = operation[2]
        arr[index] = value
        update(1, 0, n - 1, index, value)
    else:
        left = operation[1]
        right = operation[2]
        result = query(1, 0, n - 1, left, right)
        print(result[2])
n = int(input())
arr = list(map(int, input().split()))
tree = [[-10**18, -10**18] for _ in range(4 * n)]
def build(node, start, end):
    if start == end:
        tree[node] = [arr[start], -10**18]
        return
    mid = (start + end) // 2
    build(node * 2, start, mid)
    build(node * 2 + 1, mid + 1, end)
    merge(node)
def merge(node):

```

```

left = tree[node * 2]
right = tree[node * 2 + 1]
values = [
    left[0],
    left[1],
    right[0],
    right[1]
]
values.sort(reverse=True)
tree[node] = [values[0], values[1]]
def update(node, start, end, index, value):
    if start == end:
        tree[node] = [value, -10**18]
        return
    mid = (start + end) // 2
    if index <= mid:
        update(node * 2, start, mid, index, value)
    else:
        update(node * 2 + 1, mid + 1, end, index, value)
    merge(node)
def query(node, start, end, left, right):
    # No overlap
    if right < start or end < left:
        return [-10**18, -10**18]
    if left <= start and end <= right:
        return tree[node]
    mid = (start + end) // 2
    a = query(node * 2, start, mid, left, right)

```

```

b = query(node * 2 + 1, mid + 1, end, left, right)
values = [a[0], a[1], b[0], b[1]]
values.sort(reverse=True)
return [values[0], values[1]]
build(1, 0, n - 1)
q = int(input())
for _ in range(q):
    operation = list(map(int, input().split()))
    if operation[0] == 1:
        index = operation[1]
        value = operation[2]
        arr[index] = value
        update(1, 0, n - 1, index, value)
    else:
        left = operation[1]
        right = operation[2]
        result = query(1, 0, n - 1, left, right)
        print(result[0] + result[1])

```

8th Question:

```

n = int(input())
arr = list(map(int, input().split()))
tree = [[-10**18, -10**18] for _ in range(4 * n)]
def build(node, start, end):
    if start == end:
        tree[node][0] = arr[start]
        return
    mid = (start + end) // 2
    build(node * 2, start, mid)

```

```

    build(node * 2 + 1, mid + 1, end)

    tree[node] = combine(tree[node * 2], tree[node * 2 + 1])

def combine(a, b):
    values = [a[0], a[1], b[0], b[1]]
    first = max(values)
    values.remove(first)
    second = max(values)
    return [first, second]

def update(node, start, end, index, value):
    if start == end:
        tree[node][0] = value
        tree[node][1] = -10**18
        return

    mid = (start + end) // 2
    if index <= mid:
        update(node * 2, start, mid, index, value)
    else:
        update(node * 2 + 1, mid + 1, end, index, value)
    tree[node] = combine(tree[node * 2], tree[node * 2 + 1])

def query(node, start, end, left, right):
    if right < start or end < left:
        return [-10**18, -10**18]
    if left <= start and end <= right:
        return tree[node]

    mid = (start + end) // 2
    a = query(node * 2, start, mid, left, right)
    b = query(node * 2 + 1, mid + 1, end, left, right)

```

```

        return combine(a, b)
    build(1, 0, n - 1)
    q = int(input())
    for _ in range(q):
        operation = list(map(int, input().split()))
        if operation[0] == 1:
            index = operation[1]
            value = operation[2]
            arr[index] = value
            update(1, 0, n - 1, index, value)
        else:
            left = operation[1]
            right = operation[2]
            result = query(1, 0, n - 1, left, right)

    print(result[0] + result[1])

```

9th question:

```

n = int(input())
arr = list(map(int, input().split()))
first = {}
last = {}
for i in range(n):
    if arr[i] not in first:
        first[arr[i]] = i
    last[arr[i]] = i
def lower_bound(value):
    left = 0
    right = n

```

```
while left < right:
    mid = (left + right) // 2
    if arr[mid] < value:
        left = mid + 1
    else:
        right = mid
return left
```

```
def upper_bound(value):
    left = 0
    right = n
    while left < right:
        mid = (left + right) // 2
        if arr[mid] <= value:
            left = mid + 1
    else:
        right = mid
    return left
```

```
q = int(input())
```

```
for _ in range(q):
```

```
    L, R = map(int, input().split())
```

```
    answer = 0
```

```
    i =
```

```
    while i <= R:
```

```
        value = arr[i]
```

```
        start = first[value]
```

```
        end = last[value]
```

```
        actual_start = max(L, start)
```

```
        actual_end = min(R, end)
```

```

count = actual_end - actual_start + 1
if count > answer:
    answer = count
i = end + 1
print(answer)

```

10th question:

```

s = input().strip()
n = len(s)
tree = [[0, 0, 0] for _ in range(4 * n)]
def build(node, start, end):
    if start == end:
        if s[start] == '(':
            tree[node] = [1, 0, 0]
        else:
            tree[node] = [0, 1, 0]
    return
    mid = (start + end) // 2
    build(node * 2, start, mid)
    build(node * 2 + 1, mid + 1, end)
    tree[node] = combine(tree[node * 2], tree[node * 2 + 1])
def combine(a, b):
    matched = min(a[0], b[1])
    open_count = a[0] + b[0] - matched
    close_count = a[1] + b[1] - matched
    pairs = a[2] + b[2] + matched
    return [open_count, close_count, pairs]
def query(node, start, end, left, right):
    if right < start or end < left:

```

```
        return [0, 0, 0]
    if left <= start and end <= right:
        return tree[node]
    mid = (start + end) // 2
    a = query(node * 2, start, mid, left, right)
    b = query(node * 2 + 1, mid + 1, end, left, right)
    return combine(a, b)
build(1, 0, n - 1)
q = int(input())
for _ in range(q):
    L, R = map(int, input().split())
    result = query(1, 0, n - 1, L, R)
    print(result[2] * 2)
```